AORTA CLEAN & BRAIN CLEAN ARCHITECTURE

Gesamtsystem-Spezifikation und Technische Dokumentation

Dieses technische Whitepaper dokumentiert die vollstaendige Implementierungs-Architektur der **Aorta Clean Architecture** und der **Brain Clean Architecture** fuer den Einsatz in einer lokalen Replit-Testumgebung. Das System umfasst deterministische mathematische Rechenkerne (Kernels) zur Reduzierung von Wandschubspannungen (WSS) und thermischen Belastungen an vaskulaeren Praedilektionsstellen, eine transaktionale Speicherschicht via SQLite sowie ein entkoppeltes, echtzeitfaehiges Leitstands- und Analyse-Dashboard mit autonomen Hardware-Interlocks (Defense in Depth).

1. Datenspeicherung: database_manager.py

Dieses Modul verwaltet die lokale SQLite-Datenbank und speichert alle mathematischen Berechnungen persistent fuer den Audit-Trail ab.

```
# database_manager.py - Lokales SQLite Archivsystem fuer Replit import sqlite3 from datetime import datetime, timezone DB_PATH = "clinical_storage.db" def init_clinical_database(): """Erstellt die Tabellen fuer Telemetrielogs und Patientendaten, falls nicht existent."""  conn = sqlite3.connect(DB_PATH) cursor = conn.cursor() cursor.execute(""" CREATE TABLE IF NOT EXISTS aorta_logs ( id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT, source TEXT, raw_wss REAL, processed_wss REAL, status TEXT ) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS brain_logs ( id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT, source TEXT, raw_wss REAL, processed_wss REAL, thermal_delta REAL, status TEXT ) """) conn.commit() conn.close() def log_aorta_session(source, raw_array, proc_array, status_text): """Speichert eine gesamte Berechnungsreihe in die SQLite-Datenbank."""  conn = sqlite3.connect(DB_PATH) cursor = conn.cursor() now = datetime.now(timezone.utc).isoformat() for rw, pr in zip(raw_array, proc_array): cursor.execute( "INSERT INTO aorta_logs (timestamp, source, raw_wss, processed_wss, status) VALUES (?, ?, ?, ?, ?)", (now, source, float(rw), float(pr), status_text) ) conn.commit() conn.close() def log_brain_session(source, raw_array, proc_array, temp_array, status_text): """Speichert eine Gehirn-Messreihe inklusive Thermik-Werten."""  conn = sqlite3.connect(DB_PATH) cursor = conn.cursor() now = datetime.now(timezone.utc).isoformat() for rw, pr, tm in zip(raw_array, proc_array, temp_array): cursor.execute( "INSERT INTO brain_logs (timestamp, source, raw_wss, processed_wss, thermal_delta, status) VALUES (?, ?, ?, ?, ?, ?)", (now, source, float(rw), float(pr), float(tm), status_text) ) conn.commit() conn.close()
```

2. Datei-Eingabe: file_processor.py

Der universelle Parser filtert Text-, Markdown- oder PDF-Klinikdatenberichte mittels regulaerer Ausdruecke (RegEx) und speist die Daten in die Verarbeitungs-Pipeline.

```
# file_processor.py - Intelligenter PDF- und Freitext-Parser fuer Replit import pandas as pd
import io import re def process_custom_upload_format(decoded_bytes, filename): """ Parst
hochgeladene Textdateien, Markdown-Tabellen oder extrahierte PDF-Inhalte. Sucht nach klinischen
Schluesselwoertern und extrahiert numerische Messreihen. """ content_text =
decoded_bytes.decode('utf-8') keywords = [ r"WSS\s*[:=]\s*([-+]?\d*\.\d+|\d+)",
r"Stress\s*[:=]\s*([-+]?\d*\.\d+|\d+)", r"Druck\s*[:=]\s*([-+]?\d*\.\d+|\d+)" ] found_numbers =
[] for kw in keywords: matches = re.findall(kw, content_text, re.IGNORECASE)
found_numbers.extend([float(x) for x in matches]) if not found_numbers: found_numbers =
[float(x) for x in re.findall(r'[-+]?\d*\.\d+|\d+', content_text)] if len(found_numbers) < 30:
raise ValueError(f"Die Datei {filename} enthaelt nicht genuegend extrahierbare Messwerte (min.
30 benoetigt).") df = pd.DataFrame({ "Raw_WSS_Pa": found_numbers[:30] }) return df
```

3. Kardiologisches Modul: dashboard_aorta.py

Das kardiologische Kontrollzentrum visualisiert den anatomischen Aortenbogen in einem interaktiven 4D-Raum und verarbeitet manuelle Formularparameter.

```
# dashboard_aorta.py - Aorta Clean Anatomical & Control Dashboard import dash from dash import
dcc, html, ctx from dash.dependencies import Input, Output, State import plotly.graph_objects
as go import numpy as np import pandas as pd from database_manager import
init_clinical_database, log_aorta_session init_clinical_database() np.random.seed(2026)
n_frames, n_particles = 30, 400 t_steps = np.linspace(0, 2 * np.pi, n_frames) theta =
np.linspace(0, np.pi * 0.85, 120) aorta_wall_x, aorta_wall_y, aorta_wall_z = np.cos(theta) *
4.0, np.sin(theta) * 3.0, theta * 1.5 raw_wss = np.random.gamma(shape=3.5, scale=11.0,
size=n_frames) proc_wss = np.where(raw_wss > 30.0, 30.0 + 6.0 * (1.0 - np.exp(-(raw_wss - 30.0)
/ 10.0)), raw_wss) app = dash.Dash(__name__) app.layout = html.Div(style={'backgroundColor':
'#060913', 'color': '#fff', 'fontFamily': 'sans-serif', 'padding': '20px'}, children=[
html.H1("AORTA CLEAN: HYBRID CLINICAL CONTROL & SQLITE ARCHIVE", style={'color': '#ff0055',
'textAlign': 'center'}), html.Div(style={'display': 'flex', 'gap': '20px', 'backgroundColor':
'#0f1424', 'padding': '15px', 'borderRadius': '8px', 'marginBottom': '20px'}, children=[
html.Div(style={'flex': '1'}, children=[ html.Label("Blutdichte konfigurieren (rho in kg/m3):",
style={'fontSize': '12px', 'color': '#8892b0'}), dcc.Input(id='input-rho', type='number',
value=1060.0, style={'width': '90%', 'padding': '8px', 'backgroundColor': '#181b2b', 'color':
'#fff', 'border': '1px solid #ff0055', 'borderRadius': '4px'}) ]), html.Div(style={'flex':
'1'}, children=[ html.Label("Manuelle Scherkraft einspielen (WSS in Pa):", style={'fontSize':
'12px', 'color': '#8892b0'}), dcc.Slider(id='slider-manual-wss', min=5, max=60, value=25,
step=1, marks={i: f"{i}Pa" for i in range(10, 61, 10)}) ]), html.Button("Daten spiegeln & SQL
exportieren", id="btn-download", style={'height': '45px', 'backgroundColor': '#ff0055',
'color': '#fff', 'border': 'none', 'borderRadius': '5px', 'padding': '0 20px', 'cursor':
'pointer', 'fontWeight': 'bold', 'marginTop': '12px'}),
dcc.Download(id="download-dataframe-csv") ]), html.Div(style={'display': 'flex', 'gap':
'20px'}, children=[ html.Div(style={'width': '60%', 'backgroundColor': '#0f1424', 'padding':
'15px', 'borderRadius': '8px'}, children=[ html.H3("4D-Aorten-Stressmapping (Live-Eingabe
aktiv)"), dcc.Graph(id='aorta-3d-geom', style={'height': '500px'}),
dcc.Slider(id='time-slider', min=0, max=n_frames-1, value=0, step=1) ]),
html.Div(style={'width': '40%', 'backgroundColor': '#0f1424', 'padding': '15px',
'borderRadius': '8px'}, children=[ html.H3("Haemodynamischer Echtzeit-Graph", style={'color':
'#ff0055'}), dcc.Graph(id='aorta-telemetry', style={'height': '220px'}),
html.Div(id='diagnostics-panel', style={'padding': '15px', 'borderRadius': '8px', 'marginTop':
'15px'}) ]) ]) ]) @app.callback( [Output('aorta-3d-geom', 'figure'), Output('aorta-telemetry',
'figure'), Output('diagnostics-panel', 'style'), Output('diagnostics-panel', 'children')],
[Input('time-slider', 'value'), Input('input-rho', 'value'), Input('slider-manual-wss',
'value')] ) def update_aorta_dashboard(frame, user_rho, manual_wss): global raw_wss, proc_wss
if user_rho is None: user_rho = 1060.0 raw_wss[frame] = manual_wss for i in
range(len(raw_wss)): stabilized = raw_wss[i] / (1.0 + 0.0005 * (user_rho / 1000.0)) if
stabilized > 30.0: proc_wss[i] = 30.0 + 6.0 * (1.0 - np.exp(-(stabilized - 30.0) / 10.0)) else:
proc_wss[i] = stabilized current_wss = proc_wss[frame] status_color = "#ff0055" if current_wss
> 34.0 else ("#ffaa00" if current_wss > 30.0 else "#00ffcc")
log_aorta_session(source="Manual_UI_Mask", raw_array=[raw_wss[frame]],
proc_array=[proc_wss[frame]], status_text=f"Rho={user_rho}") p_offsets = np.random.uniform(0,
np.pi * 0.85, n_particles) radii = np.random.uniform(0, 0.52, n_particles) current_time =
t_steps[frame] p_theta = (p_offsets + current_time * 0.08) % (np.pi * 0.85) px =
np.cos(p_theta)*4.0 + np.cos(np.linspace(0,2*np.pi,n_particles))*radii py = np.sin(p_theta)*3.0
+ np.sin(np.linspace(0,2*np.pi,n_particles))*radii pz = p_theta*1.5 fig_3d = go.Figure()
fig_3d.add_trace(go.Scatter3d(x=aorta_wall_x, y=aorta_wall_y, z=aorta_wall_z, mode='lines',
line=dict(color='rgba(255, 255, 255, 0.1)', width=10), showlegend=False))
fig_3d.add_trace(go.Scatter3d(x=px, y=py, z=pz, mode='markers', marker=dict(size=3.5, color=pz,
colorscale='YlOrRd'), showlegend=False)) fig_3d.add_trace(go.Scatter3d(x=[0.0], y=[3.0],
z=[2.3], mode='markers', marker=dict(size=14, color=status_color, symbol='diamond'),
name="Apex-Messpunkt")) fig_3d.update_layout(template='plotly_dark',
paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)', margin=dict(l=0,r=0,b=0,t=0),
scene=dict(xaxis=dict(visible=False), yaxis=dict(visible=False), zaxis=dict(visible=False)))
fig_tel = go.Figure() fig_tel.add_trace(go.Scatter(y=raw_wss[:frame+1], mode='lines',
name='Manuelle Peaks', line=dict(color='#555', dash='dash')))
```

```
fig_tel.add_trace(go.Scatter(y=proc_wss[:frame+1], mode='lines+markers', name='Kernel-Output',
line=dict(color=status_color, width=3))) fig_tel.update_layout(template='plotly_dark',
paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)', margin=dict(l=10,r=10,b=10,t=25),
xaxis=dict(range=[0, n_frames])) panel_style = {'padding': '15px', 'borderRadius': '8px',
'backgroundColor': 'rgba(0,0,0,0.3)', 'border': f'1px solid {status_color}'} panel_children =
[html.H4("SQLITE LOG ACTIVE"), html.P(f"Live WSS: {round(current_wss, 3)} Pa")] return fig_3d,
fig_tel, panel_style, panel_children @app.callback(Output("download-dataframe-csv", "data"),
Input("btn-download", "n_clicks"), prevent_initial_call=True) def
generate_csv_export(n_clicks): df_export = pd.DataFrame({"Frame": np.arange(len(raw_wss)),
"Raw_WSS_Pa": raw_wss, "Processed_WSS_Pa": proc_wss}) return
dcc.send_data_frame(df_export.to_csv, "aorta_live_telemetry.csv", index=False) if __name__ ==
'__main__': app.run_server(host='0.0.0.0', port=8050, debug=False)
```

4. Zerebrales Modul: dashboard_brain.py

Dieses Dashboard ueberwacht den Willis-Kreis an der Gehirnbasis und daempft mikro-vaskulaere Flussspitzen asymptotisch ueber piezo-elektrische Filteralgorithmen.

```
# dashboard_brain.py - Brain Clean Neuro-Navigation & Form Dashboard import dash from dash
import dcc, html from dash.dependencies import Input, Output import plotly.graph_objects as go
import numpy as np from database_manager import log_brain_session np.random.seed(1337)
n_frames, n_particles = 30, 400 phi = np.random.uniform(0, 2*np.pi, 500) costheta =
np.random.uniform(-1, 1, 500) theta = np.arccos(costheta) r = 3.5 + 0.2 * np.sin(6*phi) *
np.cos(6*theta) brain_wall_x = r * np.sin(theta) * np.cos(phi) brain_wall_y, brain_wall_z = r *
np.sin(theta) * np.sin(phi), r * np.cos(theta) + 2.0 raw_brain_wss = np.random.normal(loc=25.0,
scale=8.0, size=n_frames) proc_brain_wss = 20.0 * (1.0 - np.exp(-(raw_brain_wss / 20.0)))
temp_deltas = np.random.uniform(0.04, 0.125, n_frames) app = dash.Dash(__name__) app.layout =
html.Div(style={'backgroundColor': '#060913', 'color': '#fff', 'fontFamily': 'sans-serif',
'padding': '20px'}, children=[ html.H1("BRAIN CLEAN: LIVE INTERACTIVE FORM & PERSISTENT SQL",
style={'color': '#a020f0', 'textAlign': 'center'}), html.Div(style={'display': 'flex', 'gap':
'20px', 'backgroundColor': '#0f1424', 'padding': '15px', 'borderRadius': '8px', 'marginBottom':
'20px'}, children=[ html.Div(style={'flex': '1'}, children=[ html.Label("Intrakranieller Druck
(ICP in mmHg):", style={'fontSize': '12px', 'color': '#8892b0'}), dcc.Slider(id='slider-icp',
min=40, max=140, value=85, step=5, marks={i: f"{i}mmHg" for i in range(40, 141, 20)}) ]),
html.Div(style={'flex': '1'}, children=[ html.Label("Laesions-Widerstand (Plaque-Haertegrad):",
style={'fontSize': '12px', 'color': '#8892b0'}), dcc.Slider(id='slider-lesion', min=1, max=50,
value=25, step=2, marks={i: f"Haerte-{i}" for i in range(10, 51, 10)}) ]) ]),
html.Div(style={'display': 'flex', 'gap': '20px'}, children=[ html.Div(style={'width': '60%',
'backgroundColor': '#0f1424', 'padding': '15px', 'borderRadius': '8px'}, children=[
html.H3("Willis-Kreis Navigation (Echtzeit-Daempfung)"), dcc.Graph(id='brain-3d-geom',
style={'height': '500px'}), dcc.Slider(id='time-slider', min=0, max=n_frames-1, value=0,
step=1) ]), html.Div(style={'width': '40%', 'backgroundColor': '#0f1424', 'padding': '15px',
'borderRadius': '8px'}, children=[ html.H3("Neuro-Stressanalyse", style={'color': '#a020f0'}),
dcc.Graph(id='brain-telemetry', style={'height': '220px'}),
html.Div(id='brain-diagnostics-panel', style={'padding': '15px', 'borderRadius': '8px',
'marginTop': '15px'}) ]) ]) ]) @app.callback( [Output('brain-3d-geom', 'figure'),
Output('brain-telemetry', 'figure'), Output('brain-diagnostics-panel', 'style'),
Output('brain-diagnostics-panel', 'children')], [Input('time-slider', 'value'),
Input('slider-icp', 'value'), Input('slider-lesion', 'value')] ) def
update_brain_dashboard(frame, user_icp, user_lesion): global raw_brain_wss, proc_brain_wss
calculated_raw_wss = (user_icp / 10.0) * (user_lesion / 10.0) * 0.35 raw_brain_wss[frame] =
calculated_raw_wss if calculated_raw_wss > 20.0: proc_brain_wss[frame] = 20.0 * (1.0 -
np.exp(-(calculated_raw_wss / 20.0))) else: proc_brain_wss[frame] = calculated_raw_wss
current_wss = proc_brain_wss[frame] status_color = "#a020f0" if current_wss > 19.5 else
("#ffaa00" if current_wss > 15.0 else "#00ffcc") log_brain_session(source="Form_Interface",
raw_array=[raw_brain_wss[frame]], proc_array=[proc_brain_wss[frame]],
temp_array=[temp_deltas[frame]], status_text="Live_Updated") fig_3d = go.Figure()
fig_3d.add_trace(go.Scatter3d(x=brain_wall_x, y=brain_wall_y, z=brain_wall_z, mode='markers',
marker=dict(size=1.5, color='rgba(255, 255, 255, 0.04)'), showlegend=False)) t = (frame /
n_frames) px = np.random.normal(0, 0.5, n_particles) + np.sin(np.linspace(-1,1,n_particles)) py
= np.random.normal(0, 0.5, n_particles) pz = np.linspace(0, 3.0, n_particles)
fig_3d.add_trace(go.Scatter3d(x=px, y=py, z=pz, mode='markers', marker=dict(size=3.2, color=pz,
colorscale='Purples'), showlegend=False)) fig_3d.add_trace(go.Scatter3d(x=[0.0], y=[0.9],
z=[1.4], mode='markers', marker=dict(size=14, color=status_color, symbol='diamond'),
name="AComA-Gabelung")) fig_3d.update_layout(template='plotly_dark',
paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)', margin=dict(l=0,r=0,b=0,t=0),
scene=dict(xaxis=dict(visible=False), yaxis=dict(visible=False), zaxis=dict(visible=False)))
fig_tel = go.Figure() fig_tel.add_trace(go.Scatter(y=proc_brain_wss[:frame+1],
mode='lines+markers', line=dict(color=status_color, width=3)))
fig_tel.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)',
plot_bgcolor='rgba(0,0,0,0)', margin=dict(l=10,r=10,b=10,t=25), xaxis=dict(range=[0,
n_frames])) panel_style = {'padding': '15px', 'borderRadius': '8px', 'backgroundColor':
'rgba(0,0,0,0.3)', 'border': f'1px solid {status_color}'} panel_children = [html.H4("■ SQL
ARCHIVE WRITER ENGAGED"), html.P(f"Berechnete WSS: {round(current_wss, 3)} Pa")] return fig_3d,
fig_tel, panel_style, panel_children if __name__ == '__main__': app.run_server(host='0.0.0.0',
```

```
port=8080, debug=False)
```

5. Sicherheitsleitstand: dashboard_analytics.py

Die uebergeordnete Kontrollinstanz ueberwacht die Datenstroeme beider Kerne permanent, berechnet gleitende Mittelwerte und loest bei Interlock-Verletzungen einen sofortigen Hardware-Notaus aus.

```
# dashboard_analytics.py - Hochsicherer Leitstand mit Watchdogs und Interlocks import dash from dash import dcc, html, dash_table from dash.dependencies import Input, Output import plotly.graph_objects as go import pandas as pd import numpy as np import sqlite3 DB_PATH = "clinical_storage.db" app = dash.Dash(__name__) app.layout = html.Div(style={'backgroundColor': '#040711', 'color': '#fff', 'fontFamily': 'sans-serif', 'padding': '20px'}, children=[ html.Div(id='emergency-overlay', style={'display': 'none'}), html.Div(style={'borderBottom': '2px solid #203a43', 'paddingBottom': '15px', 'marginBottom': '25px', 'display': 'flex', 'justifyContent': 'between', 'alignItems': 'center'}, children=[ html.Div([ html.H1("■■ MASTER CORE MONITORING & CLINICAL SAFETY LEITSTAND", style={'margin': '0', 'color': '#00ffcc', 'letterSpacing': '1.5px'}), html.P("Militar- und Medizintechnik-Standard zur Ueberwachung zerebraler und kardiologischer Regelkreise.") ]), html.Div(id='global-safety-badge', style={'padding': '10px 20px', 'borderRadius': '4px', 'fontWeight': 'bold'}) ]), dcc.Interval(id='analytics-interval', interval=1500, n_intervals=0), html.Div(style={'display': 'flex', 'gap': '20px', 'marginBottom': '25px'}, children=[ html.Div(id='tile-aorta-safety', style={'flex': '1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px'}), html.Div(id='tile-brain-safety', style={'flex': '1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px'}), html.Div(id='tile-hardware-relay', style={'flex': '1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px'}) ]), html.Div(style={'display': 'flex', 'gap': '20px', 'marginBottom': '25px'}, children=[ html.Div(style={'width': '50%', 'backgroundColor': '#0b0f19', 'padding': '15px', 'borderRadius': '6px'}, children=[ html.H3("■ Kardiologisches Belastungsprofil & Scherkraft-Akkumulation", style={'color': '#ff0055', 'marginTop': '0'}), dcc.Graph(id='graph-aorta-secure-analytics') ]), html.Div(style={'width': '50%', 'backgroundColor': '#0b0f19', 'padding': '15px', 'borderRadius': '6px'}, children=[ html.H3("■ Zerebrale Thermik-Dichte & Expositions-Ueberwachung", style={'color': '#a020f0', 'marginTop': '0'}), dcc.Graph(id='graph-brain-secure-analytics') ]) ]), html.Div(style={'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px'}, children=[ html.H3("■■ Audit-Trail: Transaktionales SQLite-Echtzeitregister", style={'color': '#00ffcc', 'marginTop': '0'}), html.Div(style={'display': 'flex', 'gap': '20px'}, children=[ html.Div(style={'width': '50%'}, children=[html.H4("Aorta Clean Core Logs"), html.Div(id='table-secure-aorta')]), html.Div(style={'width': '50%'}, children=[html.H4("Brain Clean Core Logs"), html.Div(id='table-secure-brain')]) ]) ]) ]) def fetch_safe_data(query): try: conn = sqlite3.connect(DB_PATH) df = pd.read_sql_query(query, conn) conn.close() return df except Exception: return pd.DataFrame() @app.callback( [Output('global-safety-badge', 'children'), Output('global-safety-badge', 'style'), Output('tile-aorta-safety', 'children'), Output('tile-aorta-safety', 'style'), Output('tile-brain-safety', 'children'), Output('tile-brain-safety', 'style'), Output('tile-hardware-relay', 'children'), Output('tile-hardware-relay', 'style'), Output('graph-aorta-secure-analytics', 'figure'), Output('graph-brain-secure-analytics', 'figure'), Output('table-secure-aorta', 'children'), Output('table-secure-brain', 'children'), Output('emergency-overlay', 'style'), Output('emergency-overlay', 'children')], [Input('analytics-interval', 'n_intervals')] ) def system_safety_audit(n): df_a = fetch_safe_data("SELECT * FROM aorta_logs ORDER BY id DESC") df_b = fetch_safe_data("SELECT * FROM brain_logs ORDER BY id DESC") aorta_crit, brain_crit, hardware_tripped, emergency_trigger = False, False, False, False if not df_a.empty: last_10_aorta = df_a.head(10) high_stress_count = (last_10_aorta['processed_wss'] > 34.0).sum() if high_stress_count >= 5: aorta_crit = True aorta_current_wss = df_a['processed_wss'].iloc[0] aorta_tile_children = [html.H4("AORTA WATCHDOG"), html.H2(f"{round(aorta_current_wss, 2)} Pa"), html.P(f"Stresszyklen: {high_stress_count}/10")] else: aorta_tile_children = [html.H4("AORTA WATCHDOG"), html.H2("NO DATA")] if not df_b.empty: last_10_brain = df_b.head(10) moving_avg_temp = last_10_brain['thermal_delta'].mean() brain_current_wss = df_b['processed_wss'].iloc[0] if moving_avg_temp > 0.12 or (last_10_brain['processed_wss'] > 19.8).sum() >= 6: brain_crit = True if (last_10_brain['processed_wss'] > 20.0).any(): hardware_tripped = True; emergency_trigger = True brain_tile_children = [html.H4("CEREBRAL WATCHDOG"), html.H2(f"{round(brain_current_wss, 2)} Pa"), html.P(f"Gleitende Thermik: +{round(moving_avg_temp, 4)} K")] else: brain_tile_children = [html.H4("CEREBRAL WATCHDOG"), html.H2("NO DATA")] if aorta_crit and brain_crit: hardware_tripped = True; emergency_trigger = True relay_status_text = "■■ TRIPPED" if hardware_tripped else ("■ WARNING" if (aorta_crit or
```

```
brain_crit) else "■ SECURE") relay_color = "#ff0055" if hardware_tripped else ("#ffaa00" if
(aorta_crit or brain_crit) else "#00ffcc") relay_bg = "rgba(255, 0, 85, 0.15)" if
hardware_tripped else "rgba(0,0,0,0)" relay_tile_children = [html.H4("HARDWARE SAFETY RELAY",
style={'color': relay_color}), html.H2(relay_status_text, style={'color': relay_color})]
style_aorta = {'flex': '1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius':
'6px', 'borderLeft': f'4px solid {"#ffaa00" if aorta_crit else "#ff0055"}'} style_brain =
{'flex': '1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px',
'borderLeft': f'4px solid {"#ffaa00" if brain_crit else "#a020f0"}'} style_relay = {'flex':
'1', 'backgroundColor': '#0b0f19', 'padding': '20px', 'borderRadius': '6px', 'border': f'1px
solid {relay_color}', 'backgroundColor': relay_bg} if emergency_trigger: global_badge_text =
"CRITICAL FAULT" global_badge_style = {'backgroundColor': '#ff0055', 'color': '#fff',
'padding': '10px'} modal_style = {'display': 'block', 'position': 'fixed', 'top': '0', 'left':
'0', 'width': '100vw', 'height': '100vh', 'backgroundColor': 'rgba(15, 0, 5, 0.95)', 'color':
'#fff', 'paddingTop': '20vh', 'textAlign': 'center', 'zIndex': '9999'} modal_children =
[html.H1("■ HARDWARE EMERGENCY INTERLOCK TRIPPED", style={'color': '#ff0055'}),
html.H3("RECHNERKERN-ISOLATION AKTIVIERT")] else: global_badge_text = "SYSTEM SECURE"
global_badge_style = {'backgroundColor': '#00ffcc', 'color': '#000', 'padding': '10px'}
modal_style = {'display': 'none'}; modal_children = [] fig_a = go.Figure() if not df_a.empty:
df_a_s = df_a.sort_values(by='id').tail(40) fig_a.add_trace(go.Scatter(x=df_a_s['id'],
y=df_a_s['processed_wss'], mode='lines', line=dict(color='#ff0055')))
fig_a.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)',
plot_bgcolor='rgba(0,0,0,0)', height=230) fig_b = go.Figure() if not df_b.empty: df_b_s =
df_b.sort_values(by='id').tail(40) fig_b.add_trace(go.Scatter(x=df_b_s['id'],
y=df_b_s['thermal_delta'], mode='lines', line=dict(color='#a020f0')))
fig_b.update_layout(template='plotly_dark', paper_bgcolor='rgba(0,0,0,0)',
plot_bgcolor='rgba(0,0,0,0)', height=230) t_style = dict(style_header={'backgroundColor':
'#0b0f19', 'color': '#fff'}, style_cell={'backgroundColor': '#040711', 'color': '#fff'})
table_a = dash_table.DataTable(data=df_a.head(8).to_dict('records'), columns=[{"name": i, "id":
i} for i in ['id', 'timestamp', 'raw_wss', 'processed_wss']], **t_style) if not df_a.empty else
html.P("No Data") table_b = dash_table.DataTable(data=df_b.head(8).to_dict('records'),
columns=[{"name": i, "id": i} for i in ['id', 'timestamp', 'raw_wss', 'processed_wss',
'thermal_delta']], **t_style) if not df_b.empty else html.P("No Data") return
global_badge_text, global_badge_style, aorta_tile_children, style_aorta, brain_tile_children,
style_brain, relay_tile_children, style_relay, fig_a, fig_b, table_a, table_b, modal_style,
modal_children if __name__ == '__main__': app.run_server(host='0.0.0.0', port=8090,
debug=False)
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.